

Cahier des charges — Agora, le Café Littéraire

Version du document : 1.0 — généré par analyse du code source existant **Date :** 7 juillet 2026

Statut du projet : application développée et fonctionnelle (stade avancé), ce document formalise a posteriori les spécifications à partir de l'implémentation réelle.

1. Présentation générale

1.1 Contexte

Agora est un café littéraire physique qui souhaite disposer d'une vitrine numérique communautaire. L'application doit permettre aux visiteurs de découvrir l'établissement, à ses membres d'interagir avec son catalogue de livres et son actualité, et à l'équipe du café de gérer l'ensemble du contenu depuis une interface d'administration, sans intervention technique.

1.2 Concept

Le site articule trois univers complémentaires autour du lieu :

- **Le livre** : bibliothèque consultable, avis de lecteurs, box thématiques à emporter (click & collect).
- **Le café** : carte des boissons et des mets (« moments »), recettes détaillées.
- **La communauté** : agenda culturel (concerts, événements), galerie photo, espace membre, contact.

1.3 Objectifs du produit

- Donner une image attractive et vivante du café littéraire.
- Fidéliser une communauté de lecteurs via un espace personnel et un système d'avis modéré.
- Simplifier la gestion quotidienne du contenu (livres, box, carte, agenda) pour l'équipe du café via un back-office intégré, sans base de données ni serveur à administrer.
- Proposer un parcours de réservation simple pour les box de livres en click & collect.

1.4 Périmètre

Application web responsive (mobile, tablette, desktop), en français, sans back-end applicatif propre : toute la persistance et l'authentification reposent sur des services Firebase (BaaS). Le site est généré en export statique (SSG) et hébergé sur Firebase Hosting.

2. Acteurs et rôles

Trois niveaux d'accès, déterminés par le rôle stocké dans le document Firestore de l'utilisateur (`users/{uid}.role`) et non par Firebase Auth seul :

Rôle	Description	Accès
Invité (<code>guest</code>)	Visiteur non connecté	Pages publiques (accueil, bibliothèque, coffee shop, librairie, collect, gazette, à propos), lecture des avis, inscription/connexion
Membre (<code>registered</code>)	Utilisateur inscrit et connecté, ayant terminé l'onboarding	+ Espace personnel (<code>/mon_espace</code>), dépôt/modification d'avis sur un livre, réservation et libération de livres de la « book box », gestion de son profil
Administrateur (<code>admin</code>)	Membre de l'équipe du café	+ Gestion complète du contenu : livres, box de livres, book box, concerts, événements, recettes, moments, utilisateurs, modération des avis

Règle de détermination du rôle (`getUserRole`) :

- Pas d'UID Firebase → `guest` .
- UID présent et `role === "admin"` en Firestore → `admin` .
- UID présent sans rôle admin → `registered` par défaut.

Garde de route : chaque page déclare un `sessionStatus` (`guest` , `registered` ou aucun) consommé par le composant `Session` , qui redirige automatiquement :

- une page `registered` vers `/connexion` si non connecté ;
- une page `guest` (connexion, inscription) vers `/mon_espace` si déjà connecté ;
- tout utilisateur connecté n'ayant pas terminé son onboarding vers `/onboarding` , quelle que soit la page demandée.

3. Parcours et spécifications fonctionnelles par module

3.1 Page d'accueil (`/`)

- Bandeau d'en-tête (header) de présentation du café.
- Section de présentation générale du lieu.
- Aperçu des prochains événements (`EvenementView`).
- Aperçu de la « book box » (livres disponibles au don/à l'échange).

3.2 Bibliothèque (`/bibliotheque`)

- En-tête de section.

- Catalogue des livres avec :

- fiche livre (titre, auteur, description, image, année de sortie, catégorie, nombre de pages) ; - badge « coup de cœur » pour les sélections de l'équipe ; - note moyenne calculée dynamiquement à partir des avis **approuvés** (un avis sans étoile, `rating = 0`, est exclu du calcul pour ne pas fausser la moyenne) ; la note n'est affichée que si le livre a au moins un avis approuvé ; - carte avec révélation d'un panneau de détail au survol (desktop) ou au tap (mobile/tactile).

- Système d'avis (`ModalAvis`) :

- accessible aux visiteurs **et** aux membres connectés (le nom/prénom/pseudo est pré-rempli et verrouillé pour un membre connecté, saisi librement sinon) ; - notation par étoiles interactive, champ de texte libre ; - un membre déjà connecté qui a déjà laissé un avis sur ce livre le voit pré-chargé dans la modale et peut le **modifier** (mise à jour, pas de doublon) ; - tout nouvel avis est créé avec `moderationStatus = "pending"` et n'apparaît en public qu'après validation par un administrateur ; - la modification d'un avis existant ne remet pas obligatoirement son statut à `pending` (comportement actuel du code : mise à jour directe des champs sans réinitialiser la modération — point à confirmer/valider avec le métier, voir §8).

- Section « actualités / nouveautés » (`NewsView`) mettant en avant les livres récents.
- Bloc « likes » sur les livres (`LikeView`).

3.3 Coffee Shop (`/coffeeShop`)

- En-tête de section.
- **Moments** : plats/en-cas du café avec ingrédients, allergènes, prix, type, catégorie, température (chaud/froid) et image.
- **Recettes** : boissons du café avec la même structure de données (titre, type, catégorie, température, description, ingrédients, allergènes, prix, image), présentées sous forme de cartes avec modale de détail (`RecipeModalView`).

3.4 Librairie — Book Box (`/librairie`)

Fonctionnalité de **réservation de livres à emporter/échanger** (boîte à livres du café), distincte de la bibliothèque de consultation :

- Liste des livres **disponibles** et des livres **réservés**, chacun avec titre, auteur, description, image.
- Un visiteur non connecté qui tente de réserver est redirigé vers la page d'inscription.
- Un membre connecté peut réserver un livre disponible : le document passe en statut `reserved` avec l'identité du réservataire (`userId` , `displayName`).
- Un livre peut être remis à disposition (`available` , `reservedBy = null`) par :

- le membre qui l'a lui-même réservé ; - ou un administrateur (droit de libération sur n'importe quelle réservation).

- Retours utilisateur via notifications toast (succès/erreur) à chaque action.

3.5 Collect — Box thématiques (/collect)

- Présentation du concept de « click & collect » (commande d'une box en ligne, retrait en boutique — la commande/paiement effectifs se font en boutique, aucun tunnel d'achat/paiement n'est implémenté côté site).
- Catalogue des box disponibles, avec filtre par type (**Standard**, **Standard occasion**, **XXL**, ou toutes).
- Fiche détaillée de chaque box en modale (titre, description, prix, image).

3.6 Gazette (/gazette)

Agenda culturel et vie du café :

- **Concerts à venir**, triés par date croissante, avec réseaux sociaux associés (Facebook, Instagram, TikTok, YouTube, Twitter/X) et galerie d'images multiples par concert.
- **Événements culturels** à venir, même structure qu'un concert (hors galerie multi-images, une seule image).
- Sections éditoriales : mise en avant d'un événement/concert du moment (« live », « bientôt », « hard rock »/thématique).
- **Galerie photo** de la gazette (collection `gallery`), triée par date de création décroissante.

3.7 À propos (/apropos)

- En-tête de présentation.
- Présentation du lieu (carrousel/texte de présentation).
- Carte interactive de localisation du café.
- Formulaire de contact (nom, prénom, e-mail, message) — **actuellement un formulaire HTML statique sans traitement/soumission implémentée** (pas d'appel API ni d'envoi d'e-mail au clic sur « Envoyer ») : à considérer comme une maquette à finaliser (voir §8).

3.8 Authentification

- **Inscription** (/connexion/inscription) : création de compte Firebase Auth (e-mail/mot de passe) + création du profil Firestore associé (`users/{uid}`), avec envoi d'un e-mail de vérification.
- **Connexion** (/connexion) : connexion e-mail/mot de passe.
- **Mot de passe oublié** (/connexion/mot-de-passe-perdu) : envoi d'un e-mail de réinitialisation via Firebase.
- Ces trois pages sont réservées aux visiteurs non connectés (`sessionStatus = "guest"`) ; un utilisateur déjà connecté y accédant est redirigé vers `/mon_espace` .
- Messages d'erreur Firebase traduits en français (`getFirebaseErrorMessage`).

3.9 Onboarding (/onboarding)

Parcours obligatoire déclenché à la première connexion, tant que `userDocument.onboardingIsCompleted` est `false` — bloque l'accès au reste du site tant qu'il n'est pas terminé. Étapes (`onboarding-tabs`) :

1. **Bienvenue** (`welcome-step`).
2. **Avatar** (`avatar-step`) : upload de photo de profil vers Firebase Storage.
3. **Profil** (`profile-step`) : nom, nom d'affichage, pseudo, description, adresse, centres d'intérêt, style de lecture préféré.
4. **Étape finale** (`final-step`) : validation, marquage `onboardingIsCompleted = true` .

Navigation entre étapes via un pied de page dédié (précédent/suivant), avec détection de première/dernière étape.

3.10 Espace membre (/mon_espace)

- Tableau de bord personnel du membre connecté.
- Gestion du profil (informations personnelles, réseaux sociaux : Facebook, Twitter, YouTube, TikTok, Instagram, avatar).
- Ajout d'un avis sur un livre (`/mon_espace/add-avis`), même logique de modération que depuis la bibliothèque.
- Navigation latérale dédiée à l'espace personnel (`UserAccountNavigation`), sans footer public sur ces pages.

3.11 Espace administrateur (sous `/mon_espace` , accès `admin` uniquement)

Back-office complet de gestion de contenu, chaque module suivant le pattern liste + formulaire d'ajout/édition + suppression :

Route	Fonction
/mon_espace/add-book	Gestion des livres de la bibliothèque (CRUD, upload image, marquage « coup de cœur », suppression en cascade des avis et de l'image associés)
/mon_espace/add-book-box	Gestion des livres de la « book box » (réservation)
/mon_espace/add-box	Gestion des box thématiques (click & collect)
/mon_espace/add-concert	Gestion des concerts (dates, réseaux sociaux, galerie d'images)
/mon_espace/add-evenements	Gestion des événements culturels
/mon_espace/add-moment	Gestion des « moments » (mets du café)
/mon_espace/add-recipe	Gestion des recettes (boissons)
/mon_espace/add-users	Création/édition de comptes utilisateurs, attribution des rôles
/mon_espace/users	Liste et consultation des utilisateurs
admin-account-avis	Modération des avis : liste paginée (15/page), recherche texte (insensible aux accents/casse), filtre par statut (en attente / approuvé / rejeté), édition et suppression d'un avis, changement du statut de modération

Suppression en cascade : la suppression d'un livre entraîne automatiquement la suppression de son image dans Firebase Storage et de tous les avis associés dans Firestore (opération transactionnelle applicative en 3 étapes avec rapport d'erreur détaillé par étape).

4. Règles de gestion transverses

1. **Modération obligatoire** : tout avis créé (visiteur ou membre) est en statut `pending` et invisible du public tant qu'un administrateur ne l'a pas passé en `approved`.
2. **Note moyenne d'un livre** = moyenne des `rating` des avis `approved` uniquement, en excluant les avis à `rating <= 0` (absence de note).
3. **Réservation « book box »** : un livre a exactement deux états, `available` ou `reserved` ; seul le réservataire ou un admin peut le repasser à `available`.
4. **Accès conditionné à l'onboarding** : un membre ne peut utiliser aucune fonctionnalité connectée tant que son profil initial n'est pas complété.
5. **Rôle non modifiable par l'utilisateur lui-même** : le rôle est un champ du document Firestore, modifiable uniquement depuis l'espace admin (`add-users`).

6. **Traçabilité du contenu créé par les admins** : les documents (concerts, événements, box, moments, recettes, book box) portent `userId` (créateur) et `creation_date`.
7. **Uniformisation des retours d'erreur** : toutes les fonctions d'accès aux données Firebase (Auth, Firestore, Storage) retournent un objet normalisé `{ data, error }` (ou lèvent une exception pour les lectures), affiché à l'utilisateur via des notifications toast.

5. Modèle de données (Cloud Firestore)

Collection	Rôle	Champs clés
<code>users</code>	Profils utilisateurs	<code>role</code> (admin/registered), <code>name</code> , <code>displayName</code> , <code>email</code> , <code>avatar</code> , <code>onboardingIsCompleted</code> , <code>description</code> , <code>hobbies</code> , <code>styleLove</code> , réseaux sociaux
<code>books</code>	Catalogue de la bibliothèque	<code>title</code> , <code>autor</code> , <code>description</code> , <code>image</code> , <code>releaseYear</code> , <code>category</code> , <code>nb_page</code> , <code>like</code> , <code>coupDeCoeur</code>
<code>bookReviews</code>	Avis sur les livres	<code>bookId</code> , <code>bookTitle</code> , <code>bookImage</code> , <code>userId</code> , <code>firstName</code> , <code>lastName</code> , <code>pseudo</code> , <code>avatar</code> , <code>rating</code> , <code>review</code> , <code>moderationStatus</code> (pending/approved/rejected), <code>creation_date</code> , <code>updated_date</code>
<code>bookBox</code>	Livres de la boîte à livres (réservation)	<code>title</code> , <code>author</code> , <code>description</code> , <code>image</code> , <code>status</code> (available/reserved), <code>reservedBy</code> { <code>userId</code> , <code>displayName</code> }, <code>creation_date</code>
<code>boxes</code>	Box thématiques (click & collect)	<code>title</code> , <code>description</code> , <code>type</code> , <code>price</code> , <code>image</code>
<code>recipes</code>	Recettes de boissons	<code>title</code> , <code>type</code> , <code>categorie</code> , <code>temperature</code> , <code>description</code> , <code>ingredients[]</code> , <code>allergènes[]</code> , <code>price</code> , <code>image</code>
<code>moments</code>	Mets du café	mêmes champs que <code>recipes</code>
<code>concerts</code>	Concerts à venir	<code>title</code> , <code>date</code> , <code>description</code> , réseaux sociaux, <code>images[]</code> (galerie), <code>image</code>
<code>evenements</code>	Événements culturels	<code>title</code> , <code>date</code> , <code>description</code> , réseaux sociaux, <code>image</code>
<code>gallery</code>	Galerie photo de la gazette	<code>image</code> , <code>userId</code> , <code>creation_date</code>

Le stockage des fichiers (avatars, couvertures de livres, visuels d'événements/concerts) est assuré par **Firestore Storage**, référencé par URL dans les documents Firestore correspondants.

6. Architecture technique

6.1 Stack

Technologie	Rôle
Next.js 15 (Pages Router, export statique)	Framework, routage, génération statique (SSG)
React 19 / TypeScript 5	UI typée
Tailwind CSS v4	Styles utilitaires
Firebase Authentication	Authentification e-mail/mot de passe
Cloud Firestore	Base NoSQL temps réel
Firebase Storage	Stockage des médias
Firebase Hosting	Hébergement de l'export statique
React Hook Form	Gestion et validation des formulaires
Framer Motion	Animations (transitions, textes, révélations de cartes)
React Toastify	Notifications utilisateur
react-icons	Iconographie

6.2 Organisation du code

```
`` src/ ── api/ Couche d'accès aux données (Firebase Auth, Firestore, Storage) ──
config/ Configuration Firebase, messages d'erreur localisés ── context/ Contexte
global d'authentification (AuthUserContext) ── hooks/ Hooks personnalisés
(useFirebaseAuth, useToggle) ── lib/ Constantes/utilitaires transverses (rôles,
statuts de session, types de lien) ── pages/ Routes Next.js ── styles/ CSS global,
variables de thème ── types/ Interfaces TypeScript des modèles de données ── ui/ |
├─ components/ Layout, navbar, footer, session (garde de route), sidebar... | ──
design-system/ Composants UI réutilisables | └─ modules/ Modules par page (pattern
Container/View) └─ utils/ Fonctions utilitaires (traduction erreurs Firebase) ``
```

Pattern Container/View : chaque page complexe sépare la logique (récupération de données, état, actions — *Container*) de la présentation pure (props uniquement — *View*), facilitant testabilité et réutilisation.

Sécurité et autorisation : le contrôle de rôle (admin/membre/invité) est réalisé côté client (React) à partir du document Firestore de l'utilisateur. La protection réelle des données doit être garantie par les **règles de sécurité Firestore/Storage côté serveur** (non couvertes par ce document car hors du code applicatif front) — point de vigilance à valider indépendamment (voir §8).

7. Design system

Accessible en développement via `/design-system`, il centralise les composants réutilisables :

Composant	Rôle
<code>Button</code>	Bouton avec animation de texte lettre par lettre, états loading/disabled, support icônes et liens internes/externes
<code>Card</code>	Carte livre avec révélation d'un panneau au survol/tap
<code>Modal</code>	Fenêtre modale animée (fermeture ESC + overlay)
<code>ModalAvis</code>	Formulaire de dépôt/modification d'avis livre
<code>ModalEvenement</code>	Détail d'un événement/concert
<code>Avatar</code>	Photo de profil circulaire avec état de chargement
<code>Spinner</code> / <code>ScreenSpinner</code>	Indicateurs de chargement
<code>StarRating</code>	Notation en étoiles (lecture seule ou interactive)
<code>Typo</code>	Typographie centralisée (titres serif / texte courant)
<code>Input</code> / <code>Textarea</code>	Champs de formulaire intégrés à React Hook Form

Identité visuelle

Variable	Couleur	Usage
<code>primary</code>	<code>#E35336</code> (orange)	Couleur principale, accents
<code>secondary</code>	<code>#CC7722</code> (doré)	Couleur secondaire, survols
<code>tier</code>	<code>#008000</code> (vert)	Troisième niveau
<code>background</code>	<code>#353839</code> (gris foncé)	Fond de page
<code>foreground</code>	<code>#4a4a4a</code> (gris)	Éléments de surface

Typographies : Cinzel Decorative (serif, titres) et Montserrat (sans-serif, texte courant).

8. Points de vigilance et limites identifiées dans l'implémentation actuelle

Ces constats sont issus de la lecture du code et méritent un arbitrage produit/technique avant mise en production ou évolution :

1. **Formulaire de contact (/apropos)** : purement statique (`<form action="">`), aucune soumission n'est traitée — aucun e-mail n'est envoyé et aucune donnée n'est persistée. À implémenter (ex : Cloud Function d'envoi d'e-mail, ou intégration d'un service tiers).
 2. **Page Collect** : présente un catalogue de box avec filtres, mais **aucun tunnel de commande/paiement en ligne** n'existe ; la transaction reste manuelle en boutique. À clarifier si un vrai parcours d'achat est attendu.
 3. **Modification d'un avis existant** : ne réinitialise pas systématiquement le statut à `pending` après modification par son auteur — à confirmer si un avis modifié doit repasser en modération.
 4. **Autorisation côté client uniquement** : la distinction des rôles (guest/registered/admin) est gérée en React à partir de données Firestore ; la protection effective des collections dépend des règles de sécurité Firestore/Storage (non présentes dans le dépôt applicatif analysé) — à auditer séparément.
 5. **Export statique (SSG)** : certaines données étant échangées en temps réel via Firestore côté client, le contenu affiché au premier rendu (avant hydratation) peut être vide ; comportement à valider pour le référencement (SEO) des pages à fort contenu dynamique (bibliothèque, gazette).
 6. **Absence de paiement en ligne, de panier ou de compte de fidélité** : non présents dans le périmètre actuel, à considérer comme évolutions possibles (§9).
-

9. Pistes d'évolution (hors périmètre actuel)

- Paiement en ligne et suivi de commande pour les box « collect ».
 - Traitement effectif du formulaire de contact (notification e-mail à l'équipe).
 - Système de notifications (nouvel avis approuvé, nouvelle réservation, rappel d'événement).
 - Recherche et filtres avancés sur la bibliothèque (auteur, catégorie, note).
 - Historique des avis/réservations dans l'espace membre.
 - Programme de fidélité ou avantages membres.
 - Internationalisation (dossier `src/config/locales` présent mais non exploité à ce stade).
-

10. Exigences non fonctionnelles

- **Responsive design** : mobile, tablette, desktop (classes Tailwind adaptatives observées sur l'ensemble des vues).
- **Performance** : génération statique (SSG) des pages, Turbopack en développement.
- **Accessibilité** : composants de formulaire liés à des labels, gestion clavier (fermeture des modales via ESC).
- **Expérience utilisateur** : retours systématiques via notifications toast sur les actions asynchrones (succès/échec), indicateurs de chargement (spinners) pendant les appels réseau.

- **Localisation** : interface entièrement en français, messages d'erreur Firebase traduits.